# Uncertainty Signals for Handwritten Math: Initial Results

Qwen2.5-VL-3B on FERMAT

## The question

Can a model tell us when it is wrong, without seeing the answer key? We tested one idea: ask the model the same question five times, then measure how much its five answers disagree with each other (Shannon entropy). If it disagrees with itself more on the items it gets wrong, that gives us a free way to catch errors in real use, where nobody knows the right answer yet. We tested this on two tasks, reading a handwritten math solution and judging whether that solution contains a mistake.

## Setup

Qwen2.5-VL-3B on FERMAT. Five samples per prompt at temperature 0.7, plus two greedy samples to check the sampling code works. Entropy is computed only from the model's own five answers; correctness is scored separately against ground truth. AUROC tells us whether the two line up: it is how often a randomly picked wrong item had more disagreement than a randomly picked correct one, so 0.5 means no relationship. Ranges are 95% bootstrap confidence intervals. We ran 300 items, half containing a mistake and half not, with thresholds fixed in advance.

## Results

| Task | AUROC (95% CI) | Model accuracy | Verdict |
|---|---|---|---|
| Reading handwriting | **0.835** $[0.787, 0.879]$ | 0.393 | **entropy predicts error** |
| Judging mistakes | 0.522 $[0.462, 0.582]$ | 0.517 (chance = 0.500) | **no signal** |

### Reading handwriting: it works (works)

When the model cannot read the handwriting it genuinely wavers, and that wavering lines up with its mistakes. The result holds under every check we ran. Throwing away every item where parsing failed and every item with maximum disagreement still leaves AUROC 0.796 $[0.736, 0.852]$ on the remaining 206 items, and it matches an earlier 100 item pilot that gave 0.788.

A simple rule comes out of this. **When all five readings differ, the model is wrong 57 times out of 61, or 93.4%** (95% CI 86.9% to 98.4%). That catches 31% of all reading errors, and you can apply it without knowing the right answer. Two real items:

**Item A, correct answer `option d`.** The five readings: `option d`, `option d`, `option d`, `option d`, `option d`. All agree, and the model was right.

**Item B, correct answer** $A - B \neq B - A$**.** The five readings: $A - B = \{1, 3, 5\}$ and $B - A = \{8\}$; $A = B \neq B - A$; $A = B \neq B = A$; $A \neq B$; $A \neq B \neq B - A$. Genuinely messy handwriting, the model cannot settle on a reading, and it was wrong. That instability is the signal.

Only 3 of the 300 items had all five readings agreeing and were still scored wrong, and all three are scoring quirks rather than real mistakes, for example answering `option a` where the correct answer was written `\text{A}`. So 0.393 is a strict lower bound, not our best estimate: ignoring formatting gives 0.527, and accepting the answer when it appears inside the correct line gives 0.640.

Importantly, **the AUROC barely moves across all of these**: 0.835 strict, 0.785 ignoring formatting, 0.777 under the loosest rule, and 0.814 when the whole pipeline is re-run with a different LaTeX parser. The signal does not depend on how strictly we score correctness. It also holds across conditions: 0.843 on good images against 0.818 on poor ones, and 0.837 on neat handwriting against 0.814 on messy. And it beats the cruder alternative of simply counting how many different answers came back (0.790), by a paired margin of 0.045 with a 95% CI of 0.016 to 0.075.

### Turning this into a rule for when to defer to a human

The more useful way to state the result is: how accurate are we if we only answer the items we are most sure about, and hand the rest to a person?

| We answer | Items kept | Accuracy on those |
|---|---|---|
| the most confident 13% | 38 | **0.921** |
| the most confident 33% | 100 | 0.740 |
| everything | 300 | 0.393 |

A cutoff chosen on one half of the data still controls the error rate on the other half, over 1000 random splits: aiming for at most 30% error gives 13.7% in practice, with the target exceeded in 2% of splits. The guarantee holds but is cautious, so we defer more than strictly necessary. Two limits: with five samples entropy takes only seven distinct values, so the rows above are the only choices available, and certifying a tighter target needs more data than we have. Both point at the same fix, more items and more samples each.

### Judging mistakes: no signal, because the model cannot do the task (fails)

The reason matters more than the number. In FERMAT, 84.8% of solutions contain a mistake. On a sample with that same imbalance the model scored 75%, which sounds reasonable until you notice that always answering "yes there is a mistake", without looking at anything, scores 87%. The model was doing worse than that. On a balanced set the flattery disappears:

| | | |
|---|---|---|
| Accuracy on the balanced set | **0.517** | (chance = 0.500) |
| on solutions that do contain a mistake | | 0.947 |
| on solutions that are correct | | **0.087** |
| Answers "there is a mistake" | | 93% of all items |

The model has one firm opinion, that every solution contains a mistake, and it repeats it. So it almost never disagrees with itself: 42% of items had all five answers identical, and only three different entropy values appeared in the whole run. There is nothing for entropy to measure.

## What this means

Entropy measures whether a model is consistent with itself. It never looks at whether the model is right. The method only works when those two go together, meaning the model is steady when right and wavers when wrong. That holds for reading handwriting. It fails for judging mistakes, because a confidently wrong model looks exactly like a confidently right one. Both look steady.

The clearest evidence is a smaller result. On the solutions that were actually correct, AUROC is **0.239**, below 0.5, meaning entropy ran backwards. The model was most consistent exactly when it was wrong, insisting on mistakes that were not there. Low disagreement there is actively misleading.

So the headline is not that entropy fails for reasoning. It is that **this kind of uncertainty inherits the model's ability**: it flags errors for a model that knows something and says nothing about one that is guessing. We only found this by rebalancing the test set; on the original imbalanced data the problem was invisible.

## Next step

The judging result is about this model, not the method. The obvious test is a stronger model (7B, or a text-only model given the correct transcription), with one condition: check its accuracy against the 0.500 baseline first, before computing any uncertainty numbers. If it still scores near chance the negative result holds for this kind of task; if it reaches roughly 0.65 to 0.70 the question becomes answerable again and our existing code applies unchanged.

For the handwriting side the remaining gap is a comparison against a different kind of uncertainty signal, such as the model's own token probabilities, which we did not record during generation.